



Transcript

Week 6: Supervised Learning Regression

Video 1 - Supervised Learning- Regression

How do machines recognise patterns and make decisions without human intervention?

Machine learning enables systems to learn from data and make predictions. There are two main approaches: Supervised Learning, where models learn from labeled examples, and Unsupervised Learning, where patterns emerge without predefined labels. Within supervised learning, Classification is used for categorizing data, while Regression predicts continuous values.

One fundamental technique is Linear Regression, which helps in forecasting numerical outcomes. To ensure accuracy, Evaluation Metrics are used to measure a model's performance. Understanding these concepts lays the foundation for building intelligent, data-driven solutions.

Look around—you're surrounded by patterns! The weather today might hint at tomorrow's forecast, just like your favorite streaming service suggests movies based on what you have watched. Ever noticed how supermarkets arrange similar items together? Or how your photos get grouped by faces and places? These are all examples of machine learning silently working behind the scenes. But how do machines actually recognise and learn these patterns from data? Let's find out!

Machine learning is all about algorithms that learn from data. But did you know they can learn in two very different ways?

Imagine one scenario where a teacher gives you the answers, guiding you step by step.

Now, picture another where you explore and uncover patterns all on your own.

Supervised learning is like a teacher guiding a student—it learns from labeled data, meaning it knows the correct answers while training. This helps in tasks like predicting house prices or detecting spam emails. It follows a 'Data with labels → Learning from examples' approach.

In contrast, Unsupervised learning is like sorting fruits by shape and color without knowing their names. It learns from unlabeled data, finding patterns without predefined answers. This helps in tasks like customer segmentation and market analysis. It follows a 'No labels → Discovering hidden structures' approach.

How does AI recognise patterns and make predictions? Explore the power of machine learning and see it in action!



Video 2 - Supervised Learning – Types

How does AI know the price of your dream home or if that email is junk? It all comes down to two smart tricks- regression and classification!

In machine learning, supervised learning is all about working with labeled data. But did you know there are two key types of tasks?

Regression predicts continuous values, like estimating a house price or forecasting stock trends.

Meanwhile, classification sorts data into categories, like identifying whether an email is spam or recognising a handwritten number. Together, these techniques power countless AI applications, making our tech smarter every day!

Think of grocery shopping—each time you go, you can roughly predict your bill based on the number of items in your cart. Over time, you notice a pattern: more items mean a higher bill. This is the core idea behind linear regression—a method that helps predict outcomes by analysing relationships between variables. It works by drawing a line that best represents past data to make future predictions. Our goal is to find the best-fitting line that minimizes errors between predicted and actual values. Today, we'll explore this concept using real-world examples before diving into the math behind it.

Linear regression isn't just a theoretical concept—it's used everywhere to make informed decisions. Businesses forecast sales based on advertising spending, healthcare professionals estimate patient recovery times, and financial analysts predict stock prices using past trends. Engineers also apply it to assess fuel consumption based on speed and distance. By understanding this method, we can uncover valuable insights from data and make smarter predictions.

To apply linear regression correctly, four key assumptions must be met. Linearity ensures a consistent relationship between the predictor and the outcome, meaning the change in the outcome remains constant for each unit change in the predictor. Independence means that each observation is unrelated to others and does not influence them. Homoscedasticity requires the variance of errors to remain constant across all levels of the independent variable. Normality assumes that the differences between predicted and actual values follow a normal distribution. When these conditions are met, linear regression provides accurate and reliable results.

Now, before applying linear regression, have you ensured that all key assumptions—linearity, independence, homoscedasticity and normality—are met for accurate and reliable results? Think about it!

Video 3 - Regression Metrics

How can we measure if our predictions are truly accurate?

When making predictions—like estimating your car's fuel consumption for a trip—there's often a difference between the predicted and actual values. If this process were repeated for hundreds of trips, you'd need a way to measure overall accuracy. This is where regression metrics come in. They help quantify how far off predictions are from actual values, ensuring better evaluation and improvement of predictive models.

Regression metrics help us evaluate how well a regression model performs by measuring the difference between predicted and actual values. The four primary metrics used are:

First, Mean Squared Error, abbreviated as M S E. It calculates the average of the squared differences between predicted and actual values, giving more weight to larger errors.

Second, Mean Absolute Error, or M A E. This measures the average absolute difference between predicted and actual values, treating all errors equally.

Third, Root Mean Squared Error, known as R M S E. It is the square root of the mean squared error, making it more interpretable by bringing the scale back to the original units.

Lastly, R-squared, or the coefficient of determination. It indicates how well the model explains the variation in the target variable, with values closer to one meaning a better fit.

Now, let's examine each metric in detail.

Mean Squared Error, or M S E, measures the average of the squared differences between actual and predicted values. Since errors are squared, larger mistakes are penalised more heavily, making M S E sensitive to outliers. A lower M S E indicates better model performance. This metric is widely used in training machine learning models, particularly when minimising large errors is crucial, such as in financial forecasting, where significant prediction errors can have serious consequences.

Mean Absolute Error, or M A E, calculates the average of the absolute differences between actual and predicted values. Unlike M S E, it treats all errors equally, regardless of their size. Since it does not square the errors, M A E provides a more direct measure of average prediction error, making it easier to interpret. This metric is particularly useful when we need an error measure that is less influenced by outliers.

Root Mean Squared Error, or R M S E, is the square root of Mean Squared Error. Like M S E, it penalises larger errors more heavily, but the result is in the same units as the dependent variable, making it easier to interpret. This metric is widely used to assess model performance in real-world applications such as weather forecasting and energy consumption predictions.

R squared, also known as the coefficient of determination, measures how well the regression line explains the variance in the dependent variable. It ranges from zero to one. A value closer to one means the model explains most of the variance, while a value near zero



indicates poor explanatory power. However, R squared does not measure the size of prediction errors; it only assesses how well the model fits the data.

Different metrics serve different purposes. If avoiding large errors is crucial, use MSE or RMSE. For a straightforward measure of average error, MAE is a great choice. If you want to see how well the model explains data variability, R-squared is the right metric.

Pick the right metric for your model! Analyse your data, choose wisely, and optimise for better predictions.

Video 4 - Linear Regression Basics Building from Scratch - Demo

Hello and welcome everyone. In the today's demo, we are going to talk about the way linear regression algorithm is implemented and we are going to see how programmatically we will be able to find the best set of parameters which is beta naught and beta 1 which you might have seen in the discussion we had before this demo. We are all aware that the linear regression equation is y equals to beta naught plus beta 1 x where x is your input variable and y is your target variable or the dependent variable.

Before we proceed ahead with this demonstration, I have a request to all of you do not get overwhelmed with the implementation of linear regression code, it is very simple. What I want all of you to observe over here is that how algorithmically we will be able to model the relationship between the input which is x and the output which is y . So, let us start with this demo. To run this demo, we are going to import certain libraries and this is the data which we are going to use.

If you see this data, there is an x variable and there is a y variable. I have printed the first 5 rows of the x variable and the last 5 rows of both these variables. If you look into this equation, this is what the equation of linear regression is.

We just have replaced beta 1 with m and beta naught with c . In the context of coordinate geometry to make our lives easier, the error which is reported is called as the mean squared error and this is the formula for the mean squared error. The algorithm which we use to optimise the values of m and c is called as the gradient descent method and from the name itself it says that we are going to use gradients and that is the reason we have calculated the gradients with respect to your m parameter and another gradient with respect to your c parameter. This is exactly implemented here in this equation.

I am interested in how linear regression models this relationship. To do so, I am going to start with a random initialisation of both parameters. You can give it any value.

I start with 0. I set the learning rate which is one hyper parameter in the gradient descent optimisation algorithm which is basically to set the speed of training, how fast I want to train my algorithm. You cannot set a very large learning rate and we do not have to also set a very small learning rate. So, we set a learning rate of the order mentioned over here and we start with simple training and we go for around 10 iterations.



We could also go for more than 10 iterations, but we are just going to run for 1000 iterations and in each of these iterations, I am going to again find out for each 100 iteration the sum of error and the final values of my m and c and I am going to plot that. So, you see that how m and c initially started with 0 and they are updated to a new value and the error which is reported is quite high 16. In the next iteration, it reduces these values, they optimise and this happens for 10 iterations.

We reduce the error from 16 all the way to 11 and we have our final line plotted like this. For this data, clearly this does not seem to be the best fit line. We can either go for furthermore iterations or we can also increase the learning rate and we tend to increase the learning rate and then go for few more iterations, another set of 1000 iterations.

The error further reduces and then after a while you will see that the error becomes consistent. This is an indication that it cannot fit better than what these values are and when you see the line, the line tilts more and this very much looks like a proper relationship model between x and y . You can even continue that for furthermore iterations, but you will observe that the error does not reduce anymore which is an indication that whatever line you have is the best fit line. So what we observed here is that how the linear regression tries to model the relationship between an input and an output variable.

Next you would see that since this is a non-linear data, a straight line would not be possible to properly model this kind of data. We would need a non-linear relationship which would be possible only through non-linear regression and one such regression is polynomial regression. For linear regression, that is all from this demonstration, hope it clears your doubt.

Thank you everyone.

Video 5: Regression Metrics-Demo

Hello and welcome everyone. In this demo, we are going to see about Regression Metrics. We have known to use different regression metrics and here I am going to show you few of those metrics.

So, SAE stands for Sum Absolute Error, MAE stands for Mean Absolute Error, SAC stands for Sum Squared Error, MSE stands for Mean Squared Error, RMSE stands for Root Mean Squared Error and R squared is another metric. Now, I am going to demonstrate on some dummy data. We start with having 7 observations.

Let us say these are the actual values which I am planning to predict. You know that when we start with prediction, the most naive way of predicting is to predict the mean of the values. What I mean to say is that if I ask you what would be the stock price of a certain stock tomorrow, the most naive thing which you would do is you would average the stock prices of the same stock for the past 7 days and you would provide me that as a prediction.

That is what we call mean as a measure of central tendency. So, we employ the same idea over here. We have these values which we are planning to build a model and if you calculate the mean of these values, you would see it comes out to be 4.86 round figure.

So, let us say I am just using a simple model which does nothing but calculates the mean of your available values. So, now assume that I am using a mean prediction model and it



predicts me these are the values. So, these are my actual values and these are all my predicted values by my model which does nothing but just predicts the mean of the values.

Now, if I ask you the error made by this model in the most simplest terms what you would say is you would simply say the actual minus the predicted value and that is what I have done over here on the right-hand side. If you see it is nothing but the actual minus the predicted value. So, $3 - 4.86$ is going to give you -1.86 and I do the same for all the observations.

Now, if I ask you what are the total sum of errors. So, you would do nothing you would just sum them up. So, if you see here this is nothing but summing all these values, but what do we see the sum totals to 0 which is very obvious.

If you carefully think it is very obvious when you use a mean the sum would be 0. Why is it 0? Because the algebraic sum where the positives and the negatives they cancel out each other would be 0. Since, the sum is 0 the mean would also be 0. So, if this question arises in your mind that why do not we do a simple actual minus predicted this is the reason why we do not do. If you do that and you take the algebraic sum they would cancel out each other and it would misinterpret or the sum of the error would be misleading. You would assume that your model is not making an error.

In fact, the model is making huge error individually. To fix this problem a simple addition of absolute value concept solves the problem. What I do is I just take the difference and apply an absolute.

So, my negative values turn out to be positive. In a way I am just extracting the magnitude of the error and I am leaving out the sign. So, I extract all the magnitudes and then when I do a sum I do see a value 14.86 and this value is called as sum absolute error because these this is the sum of all the individual absolute errors.

And if you want to know the average error you can calculate the mean of that. So, if you take the mean of this value or if you take the mean of all these values it would come around to be 2.12 and that is what we call as a mean absolute error. Now, there is another metric called as a squared error.

If you carefully see what am I doing over here I am just doing nothing I am actually squaring this value. So, squaring this value guarantees me that all my resultant values would be positive, but what it also does is it also amplifies these values. It also amplifies these values if you carefully see.

The question is why are we doing this? The answer to this question lies in the use of error metric in machine learning. You would realise that there is an algorithm called as the gradient descent algorithm which works on the error reported by the model. The higher the error reported the more the optimisation happens within the model.

So, if you want a quick efficient learning, you would want the reported error to be amplified and that is the reason why we try to square the error. Squaring the error amplifies the error. So, if you made an error of 2.86 the error reported is 8.16. If you made an error of 2.14 the error reported is 4.59. If you made an error of 4.14 you the reported error is 17.16. So, if you see these are all the values where the error reported is amplified.



Amplified error would mean severe optimisation severe optimisation would mean quicker learning. There is one more effect of squaring the error. If you see the error which lies between 0 and 1 which is 0.86 if you square that it even becomes a smaller 0.73. So, what do we learn? We learn that if the actual error is between 0 and 1 my reported error would be very small which is very obvious.

If the model makes a small error, we try to ignore that. We do not try to apply any severe optimisation, but when the error is more than 1 and the higher it goes the severe the the optimisation should happen. And as you go beyond 1 you would see when you reach 4 the error reported reaches 70.

So, these are the two reasons why we tend to use a squared error. When you see the sum of all the errors it is also going to be magnified that is 38.86 and we call this as a sum squared error. If you take the mean of that which is 5.55 which is also more like a squared value and we call that as a mean squared error.

Now, because you have amplified it for a certain purpose you have changed the scale of error from a linear scale to a squared scale. Because you have changed the scale of the error this reported error mean squared error is not the same as of your original data because your original data is on a linear scale this is on a squared scale. To bring this back on the linear scale you take a root of that and now when you take the root of that 2.35 is now going to be on the same scale as of your data and that is what we call as a root mean squared error.

You see that all of this had been done by assuming that the model does nothing, but blindly predicts the mean of all the actual values. Now, you know that when you are building a machine learning model it is definitely going to do better than your mean prediction model and that is what I have done over here. If you see the table below these are the same actual values as on the top, but instead of using a mean I am using a machine learning model definitely a machine learning model would tend to predict values closer to the actual values.

For example, if the actual value was 3, the machine learning model predicted 2.9 and the same way you have all the other values. For each of these actual values you have a corresponding value predicted by the machine learning model. Why are we studying this? Let us try to do a similar sort of calculation where I do an actual minus predicted, I do an absolute error, I do a squared error and I get the corresponding values.

Now, you would see that what am I going to do here is I am going to compare this value with this value and I am going to take the ratio of this value to this value over here. So, if you see this what does it say? It says that I am going to take this the sum squared error made by the mean prediction model subtract it with the sum squared error made by your ML model and divide by the sum squared error made by your mean prediction model. So, it is kind of taking the percentage improvement and what I get to see is a value 0.58. What does it mean? First of all this is what we call as an R squared value and what does it say? It says that I am 58 percent better than a mean prediction model.

Why does it say so? You see that when I was using a mean prediction model, I was getting an error of 38.86, when I am using a machine learning model, I am getting an error of 16.27. So, there is an improvement from 38 to 16, there is a reduction in error. How much of that reduction is reported by your R squared value and that is the reason why R squared value is defined. It said that how much of the variation of the target variable is explained by your



machine learning model.

The more the variation of your target variable explained by your machine learning model the lower the error, the lower the error the higher this value. If you do not believe me let me show you. Let us say these are the predictions by your machine learning model and these are your actual values.

Let me try to move these predictions even closer to the actuals. For example, this is 3 the machine learning model predicted 5, I would change this to 4. The moment I change this to 4 observe this value, how this value is increasing because I am moving closer to the actuals. Let us say this is 7, I am at 5.5, I will move this to 6.5, look into this value.

So, what is happening is as I am moving closer to the actuals my error is reducing. As my error is reducing, I am showing improvement over my static value and as I am showing improvement this improvement is quantified over here. The more the improvement the higher this value and a time would come if you actually are able to predict equal to the actuals your error would turn out to be 0 and this would turn out to be 1 which means that you are able to exactly predict the target variable and that is the reason why r squared is 1 which does not happen usually.

You would reach towards 1, but you would not exactly reach 1. So, I would just go back roll back where I was. So, that is what your r squared value does. There is a question, the question is what can be the highest and the lowest possible value of r squared metric? Think of it and do let us know when we discuss error metrics in our live sessions.

Thank you.

Video 6: Regression Assumptions- Demo

Hello everyone, in this video we are going to talk about the Fundamental Assumptions of Linear Regression Algorithm. In the past couple of videos, we have gone through the linear regression algorithm, its mathematical formulation and we also saw the regression metrics. In the today's video, we are going to talk about few of these fundamental assumptions and the first of the fundamental assumption of linear regression is linearity.

When we say linearity, we mean that it is the relationship between each independent variable which are your predictors or sometimes you call them as explanatory variables or features with data dependent variable that is your target. This relationship is linear in nature and that is the reason why mathematically if you see we assume that the mean of the target variable can be described as a straight line or a plane when you have multiple features and this is the core idea behind why we even use a linear model because we expect that as the input changes proportionally the output changes. So, if you see this is a classic equation of linear regression algorithm with multiple features where x_1, x_2 so on and so forth are your input features and w_1, w_2 are the associated weights or you can call that coefficients of linear regression algorithm and that is what we try to learn and b is the intercept and ϵ is the error term.



Now in case if the assumption is violated that is the relationship between the independent variable and target variable is not found to be linear then you will have to do transformation of your features and you might have heard about something called as feature engineering which you will do to ensure that there is a linear relationship between them before you model them. How do we detect linearity? So, we can do that by plotting our actual data points and our predicted values and if the predicted values lie close to the actual trend you can assume it to have a linear pattern. The other way to do that is to plot your residuals that is the difference between the actual and predicted values and if you see that the residuals look randomly scattered around 0 you are in good shape.

So, we have a python working example where we are randomly generating some data set x and y this is a synthetic data where in reality the relationship is linear although we have added a bit of noise, but we are going to fit a simple linear regression and plot the residuals. So, you know that this is how you split the data you fit your linear regression algorithm and then you go by plotting. So, there are two types of plot the scatter plot as I said earlier which is going to study your actual data points versus your predicted values you see your actual versus predicted values.

And the next one would be your residuals that is what we call as a residual plot. So, if you observe these two plots let us start with the first one. So, if you see the first plot you would observe that your actual data points and your predicted values, they actually lie close to the trend of this data.

If you see the trend of this data is in this direction and the predicted values also follow that trend because they follow that trend I can assume that they have a linear pattern and not only this if you see the residual plot you would see that they are randomly scattered we do not see any obvious usual curve or trend which clearly explains that your assumption of linearity holds. Had it been showing some clear pattern for example, it may be showing you a curve or a wave that is a clear indication that your relationship might be non-linear and, in that case, you might need to transform your variables or you might need to try a different model altogether. So, what we learned is that always try to test linearity by visually observing such kind of plots first and that is the most easiest way to catch the problems early in your experiment That is our first assumption and the second assumption moving ahead is the independence of errors.

Now, what this means is that the residuals which is technically the difference between the actual and the predicted values they should be independent of each other or in simple terms we can assume that or we can say that the errors from one prediction should not influence the errors from another prediction. And why does this matter? It matters that if your residuals are correlated it means that there is some pattern or information which is left in the data that your model did not capture and this leads to underestimated standard errors or this leads to some misleading inference about your data. Now, this kind of assumption is very important for time series data which you will see in the next few sessions where values naturally occur in sequence where the data is temporal in nature.

For example, think about the monthly sales or think about stock prices. So, in such kind of scenarios it is very common for errors to be correlated over time and that is very famously called as autocorrelation. So, how do we check this? Again one quick way is to plot your residuals in the order of the data points or in order of your time if you have time series data and check that if your residuals bounce randomly around 0 without any pattern that is a



good sign that they are independent.

But if you see trends for example, you see residuals consistently above 0 for a stretch and then below 0 that suggests there is autocorrelation. Another standard check is the Durbin Watson test which is a statistical test and tells you how much correlation or how much autocorrelation exists in the residuals and a value around 2 means that they are independent and no autocorrelation exist. If a value below 2 is being observed it means there is positive autocorrelation that is the error tends to repeat itself and a value above 2 means there is a negative correlation which means that errors tend to switch directions.

So, let us see our Python example again almost the very same way we are generating a simple trend and added a random noise. So, my trend plus noise is my data now. I go by fitting a linear regression model on x and y calculate the residuals simply and try to plot the residuals.

Also we try to do the Durbin Watson test and try to plot the values of the test. First let us see the chart. So, what do we have to look into this chart? So, we have to look are the residuals showing any obvious trend and if you clearly see this observe this you would see that the residuals do not show any obvious trend in the order of time that is very important to remember it should be in the order of time.

So, there is one thing which I can somewhat trust that the independence assumption holds. Now if you do find autocorrelation in the chart you might need a more advanced model like in time series we have ARIMA and if you check the Durbin Watson statistic it also comes around to be 1.84 and as I said it should approach 2. So, the result of the chart shows no residuals no trend in the residuals and the Durbin Watson is also around 2 we can clearly trust the independence assumption holds. So, the takeaway is to check your residuals they behave randomly and that is our evidence that the model has captured the structure it should right cool.

So, let us move ahead to the third assumption that is homoscedasticity. Now, this is one of the key assumptions in linear regression and although the word is tricky to pronounce, but it simply means that the variance of the residuals is constant across all the levels of your predicted values. In simple words, when I talk about the variance of the residuals I am talking about this spread of the errors and what are errors? Errors is nothing but the difference between your actual and predicted values.

So, what I am trying to say is that the spread of the errors should be roughly the same and no matter whether your predictions are large or small and why is this important? So, think if the variance of the residual changes for example, if the residuals get larger as the predicted value gets larger this is called as heteroscedasticity and when this happens the standard errors for your coefficients can be biased which means that your confidence intervals and hypothesis tests becoming unreliable. So, if I repeat it again the idea here is to ensure that the spread of the errors which is the difference between the actual and the predicted values should be roughly the same irrespective of the magnitude of the predictions. So, how do we check that? Again, the best way to do that is to plot the residual versus predicted values.

So, I again take the data and the synthetic data and intentionally create it where the variance the error increases that is an intentionally created data to show you that how the variance of the error increases as the predicted value or the predictor value increases. You will clearly

see in the plot the residuals will spread out more as the predictions get larger and it should show you a funnel shape you know picture which is a classic heteroscedasticity example. So, what are we going to do here is we are going to take this synthetically and intentionally generated data set and I am going to model linear regression algorithm and ask it to do predictions and I calculate the residuals and I simply plot the residual versus predicted values.

So, residuals versus predicted values. So, you see that on the x axis you plot the predicted values on the y axis you plot the residuals. And now the test is that if the residuals are scattered randomly and they just form some sort of a horizontal band that is good the variance is constant, but because we have intentionally created the data set like that you would not see it.

So, you will see that the residuals spread out more you see the residuals are spreading out more and as the predictions get larger these are predicted values as they get larger that spread you see this is spread this is becoming larger. Still it is not a funnel shaped, but somewhat you can say that there is a flavour of heteroscedasticity towards the right-hand side. So, what we want is we want to ensure that if in case if we see that there is a sudden spread of variance of errors, we would want to treat it by applying some simple transformation and the best one is to apply the log of the target variable, but remember that constant variance will keep your estimates more trustworthy.

So, always check for it visually and it is one of the simplest ways to spot problems in your model. And you can actually try to change these values these values you can try to change these values and you would observe the change in the output. Now, the next assumption is the normality of errors.

Now, this assumption says that the residuals which is the difference between actual and predicted values should be normally distributed, why? Strictly speaking the normality of residuals is not needed for the model to make good predictions because, linear regression can still find the best fit line using least squares even if the errors are not perfectly normal. However, normality does matter when you are constructing confidence intervals for coefficients or when you are making sure that the p values which are learned are meaningful. So, if the residuals are not normal your inferences about the model coefficients might be misleading.

So, how do we check normality? Again I kind of rely on practical checks via plots. I want to plot the histogram of residuals this time and check does it look like a bell shaped curve or there is a quantile-quantile plot a Q-Q plot which checks if the points fall roughly on the reference line that is the residuals are likely normal or there is a statistical test Shapiro-Wilk test which gives you a p value to test the null hypothesis that the residuals are normally distributed. So, let us try to implement that in Python.

So, again I have created a synthetic data of x and y. I would request you to try to change these values the y values and by changing these values observe the change in the output. So, we are going to plot this histogram of residuals you know that we use this `plt.hist`. Now, that is our histogram plot and then you have the quantile-quantile plot and then you have the test. So, if I see the histogram of the residuals what do what am I looking for? I am looking for a bell-shaped curve.

I am looking for a graph which is roughly symmetric and centred at 0. If you see this plot, it roughly is symmetric. So, it is kind of a bell-shaped curve although you would say that it is more right tail, but yeah it is roughly symmetric and it is almost centred around 0 almost. Similarly if you see the quantile-quantile plot, so what you observe here is that it should show the points lining up elegantly with the diagonal line.

So, you see the points lining up elegantly with the diagonal line. If you see the points scattered not aligned with the diagonal line that is a clear indication that the normality does not exist. Finally, you can also use the test where if the p-value is greater than 0.05 we fail to reject normality that is good and if the p-value is less than 0.5 you might need to look deeper maybe there are outliers or you need to transform your target variable.

So, here you see the p-value is 0.03 which is obviously less than 0.05. So, statistically they are asking you to be suspicious about your data and just observe that is there a need to transform your target variable. But when you look into the graph both the histogram and the Q-Q plot, so with all these three evidences you can infer that the variable follows the condition of normality. So, we need to check that the residuals look normal to be confident that your tests and intervals are valid.

Let us move to assumption number 5 that is multicollinearity. Now, this assumption says that your independent variables should not be highly correlated with each other very important. So, here I am only talking in the context of independent variables.

So, take all your independent variables and study whether they are highly correlated with each other or not. Why does this matter? This matters because when two or more than two variables are highly correlated, then there is a possibility of overlapping information about the target, redundant information about the target variable. And if there is an overlap of information, it makes it very hard for the regression to figure out the unique contribution of each predictor.

And what this results in that the model still will be able to do prediction, but the coefficient estimates would be unstable because the regression model is finding it hard to figure out the unique effect of each predictor. And you would observe that any small change in the data can lead to catastrophically large changes in the estimated coefficients. And you know that that would clearly lead to the standard errors of the coefficients to go up and eventually the p values also will become unreliable.

So, instability, unreliability are the major concerns which hurts the prediction accuracy directly. The cause is multicollinearity. So, how can you check that? Personally, I try to build a correlation matrix of our feature and see a pair of predictors with a correlation close to plus 1 or minus 1 which are extreme correlations and that is a red flag.

But since we more or less rely on numbers, so there is a way called as variance inflation factor, VIF which measures how much the variance of a coefficient is inflated because of the correlation with other predictors. So, if your VIF is 1 means no correlation, greater than 5 or 10 is a common threshold to be very careful about. So, let us try to see this Python example.



Artificial Intelligence and Machine Learning

So, I have taken actually three different predictors, x_1 , x_2 , x_3 and of these three predictors, two of them are almost identical. If you see x_1 and x_3 , they are almost the same, x_1 and x_2 , if you see x_1 and x_2 , you see this is clearly correlated with x_1 , x_2 is clearly correlated with x_1 . Now, let us try to calculate the correlation matrix and also the VIF.

So, to calculate the VIF, we are going to import variance inflation factor and just calculate the VIF for all the values. And the VIF for these predictors, if they are large that confirms multicollinearity. Let us see, I see the correlation matrix, I clearly see x_1 and x_2 , they are highly correlated 3.99 is the correlation, highly correlated.

On the other hand, see x_3 and x_1 , they are 0.01, which is the ideal scenario. Also if you see the variance inflation factor, you would see the variance inflation factor for x_1 and x_2 being very high, which means that there is multicollinearity between them. The question is, what do we do then? A very simple answer is because there are two predictors providing the same set of information, drop one of them and that is where you have these regularised linear models like ridge regression, which will help you stabilise these coefficients by adding a penalty term.

So, low multicollinearity would mean that your model can reliably tell you the real effect of each feature. So, always check it, it is easy to miss and it can really affect your interpretation at a later stage. So, these are some five major assumptions of your linear regression model. And as an experienced data scientist you should always try to check for these assumptions and if you find any of these assumptions being violated, try to deal it with the maximum possible reach to your data, to your business partners. Thank you everyone.